
CTF ЗВІТ

Capture The Flag — Звіт про виконання завдань

Курс	CyberSecurity — Практичний курс
Захід	CTF (Capture The Flag)
Платформа	https://neoversity.cybericebox.com/challenges
Завдань	12 (завдання 0–11)
Прапорів	12 / 12 (100%)
Рік	2026
Виконавець	Задорожняк Ігор

Зведена таблиця завдань

#	Назва завдання	Категорія	Складність	Прапор
0	Аналіз вихідного коду програми на Go	Source Code Analysis	Easy	ICE{Ver1_gooooD}
1	Пошук прихованих даних у HTML-кодї вебсторінки	Web / OSINT	Easy	ICE{s0urc3_h4ck3r}
2	Аналіз PCAP-файлу та дешифрування шифру Атбаш	Network Forensics / Cryptography	Medium	ICE{pacific_bell}
3	Відновлення Telnet-сесії з PCAP-файлу	Network Forensics	Easy	ICE{uk_43132}
4	Декодування Base64 у консолі браузера та знаходження прихованої сторінки	Web / JavaScript	Medium	ICE{sn4k3_v4ult_unl0ck3d}
5	Декодування фрагментів Base64+Hex та відновлення прапора з NATO-алфавіту	Network Forensics / Encoding	Hard	ICE{GHOST}
6	Аналіз TTL у ICMP-трафіку для знаходження кількості хопів	Network Forensics / TTL Analysis	Medium	ICE{13}
7	Виявлення застарілого API-endpoint та витоку токена	Web / API Security	Medium	ICE{4p1_134k}
8	Відновлення мережевого кадру з Hex-дампу та декодування токена	Forensics / Binary Analysis	Hard	ICE{fr4me_d3c0d3d}
9	Пошук облікових даних у HTTP-запиті з URL-декодуванням	Network Forensics / URL Encoding	Easy	ICE{aKuL1edo3}
10	Зворотна інженерія змінного байтового шифру (byte-shift cipher)	Cryptography / Binary Analysis	Hard	ICE{t1m3_shift}
11	Реверс-інжиніринг Python-програми: XOR + реверс + Base64	Cryptography / Reverse Engineering	Hard	ICE{Ph4nt0m_unm4sk3d_2026}

Завдання 0 — «Початок розслідування»

Категорія:	Source Code Analysis	Складність:	Easy
Інструменти:	Text Editor / VS Code		

Опис завдання

Завдання полягало у перевірці вихідного коду Golang-програми. Прапор зберігався у відкритому вигляді всередині рядкової константи, що є класичним прикладом небезпечного зберігання секретів у вихідному коді.

Покрокове виконання

1	Завантажено файл main.go та відкрито у текстовому редакторі.
2	Проведено статичний аналіз вихідного коду.
3	У коді виявлено рядок з прихованим прапором у форматі ICE{...}.
4	Прапор зчитано безпосередньо з коду без виконання програми.

ПРАПОР	ICE{Ver1_gooooD}
--------	------------------

Скріншоти

```
1 package main
2
3 import "fmt"
4
5 // Operation PHANTOM - Agent Initialization Module
6 // Build: 2025-01-15 | Version: 1.0.3
7 // Authorization token: ICE{Ver1_gooooD}
8
9 func main() {
10     fmt.Println("Initializing secure connection...")
11     fmt.Println("Loading agent profile...")
12     fmt.Println("Status: READY")
13 }
14
```

Завдання 1 — «Перший контакт»

Категорія: Web / OSINT Складність: Easy

Інструменти: Web Browser · View Page Source (Ctrl+U)

Опис завдання

Прапор зберігався у HTML-коментарі вебсторінки. Це ілюструє поширену помилку: розробники залишають відлагоджувальну або конфіденційну інформацію у клієнтській частині HTML, хоча вона невидима в інтерфейсі, але легко доступна через перегляд коду сторінки.

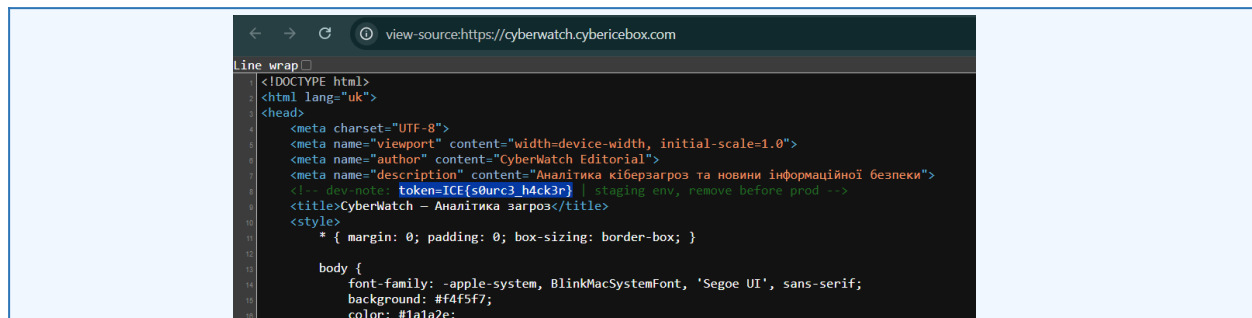
Покрокове виконання

- 1 Відкрито вебсторінку <https://cyberwatch.cybericebox.com>.
- 2 Використано функцію «Переглянути код сторінки» (Ctrl+U / View Page Source).
- 3 У HTML-коментарях виявлено рядок з прапором у форматі ICE{...}.
- 4 Прапор скопійовано та перевірено.

ПРАПОР

ICE{s0urc3_h4ck3r}

Скріншоти



Завдання 2 — «Мережевий слід»

Категорія: Network Forensics / Cryptography Складність: Medium

Інструменти: Wireshark · Atbash Cipher Decoder

Опис завдання

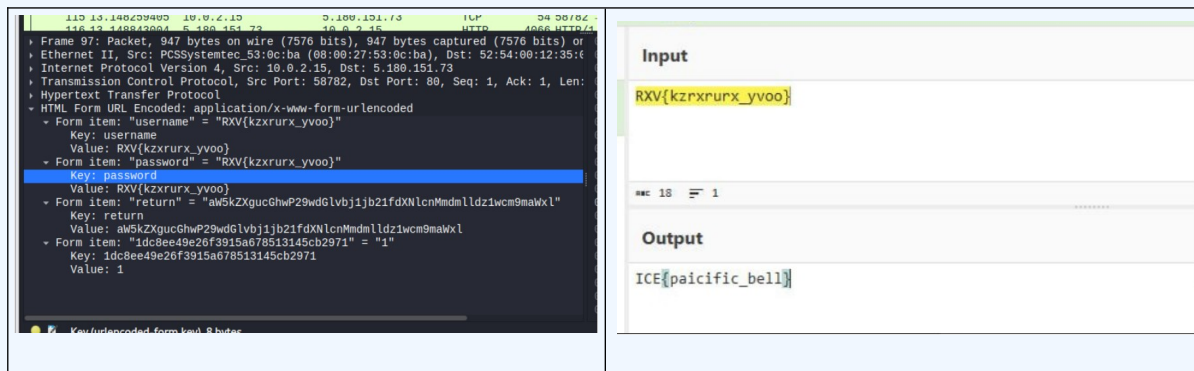
Завдання поєднало мережевий аналіз та базову криптографію. У захопленому HTTP-трафіку містився рядок, зашифрований шифром Атбаш — одним із найдавніших алфавітних шифрів підстановки, де кожна літера замінюється на «дзеркальну» ($A \leftrightarrow Z$, $B \leftrightarrow Y$ і т.д.).

Покрокове виконання

- 1 Відкрито файл f1_13.pcapng у Wireshark.
- 2 Застосовано фільтр http для пошуку HTTP-трафіку.
- 3 У POST-запиті виявлено зашифрований рядок: RXV{kzxrurx_yvoo}.
- 4 Підказка «баш» вказала на шифр Атбаш (дзеркальне відображення алфавіту).
- 5 Застосовано дешифрування Атбаш: $R \rightarrow I$, $X \rightarrow C$, $V \rightarrow E$ і т.д.
- 6 Отримано розшифрований прапор ICE{pacific_bell}.

ПРАПОР ICE{pacific_bell}

Скріншоти



Инструменти: [Wireshark](#) · [Follow TCP Stream](#)

Telnet є застарілим протоколом, який передає всі дані, включаючи паролі, у відкритому вигляді. Wireshark дозволяє повністю відновити сесію через функцію Follow TCP Stream, що робить перехоплення Telnet-сесій тривіальним завданням для злоумисника у тій самій мережі.

1	Відкрито файл f2.pcapng у Wireshark.
2	Проаналізовано протоколи через меню Statistics → Protocol Hierarchy.
3	Виявлено незашифрований Telnet-трафік.
4	Правою кнопкою на TCP-пакеті → Follow TCP Stream.
5	У відновленому потоці знайдено облікові дані та прапор.

ICE {uk 43132}

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	169	100.0	11824	1,944	0	0	0	169
Null/Loopback	100.0	169	5.7	676	111	0	0	0	169
Internet Protocol Version 4	100.0	169	28.6	3380	555	0	0	0	169
Transmission Control Protocol	100.0	169	45.9	5428	892	75	2420	398	169
Telnet	55.6	94	19.8	2340	384	94	2340	384	94

Завдання 4 — «Гра розуму»

Категорія: Web / JavaScript

Складність:

Medium

Інструменти: Web Browser · DevTools Console · Base64 (atob())

Опис завдання

Завдання демонструє техніку приховування контенту через JavaScript-логіку. Base64-кодування часто помилково сприймається як шифрування, хоча є лише форматом представлення даних. Прихована сторінка /v4ult.html була доступна без автентифікації після отримання інформації про цей існуючий шлях.

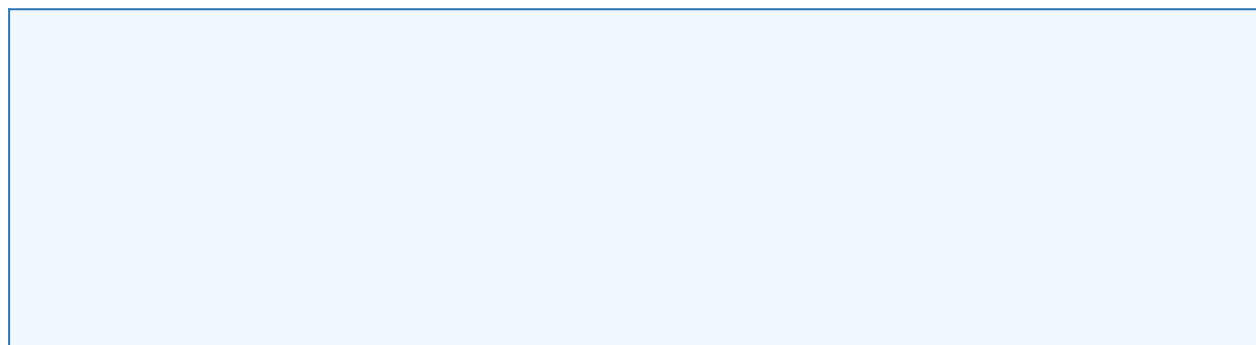
Покрокове виконання

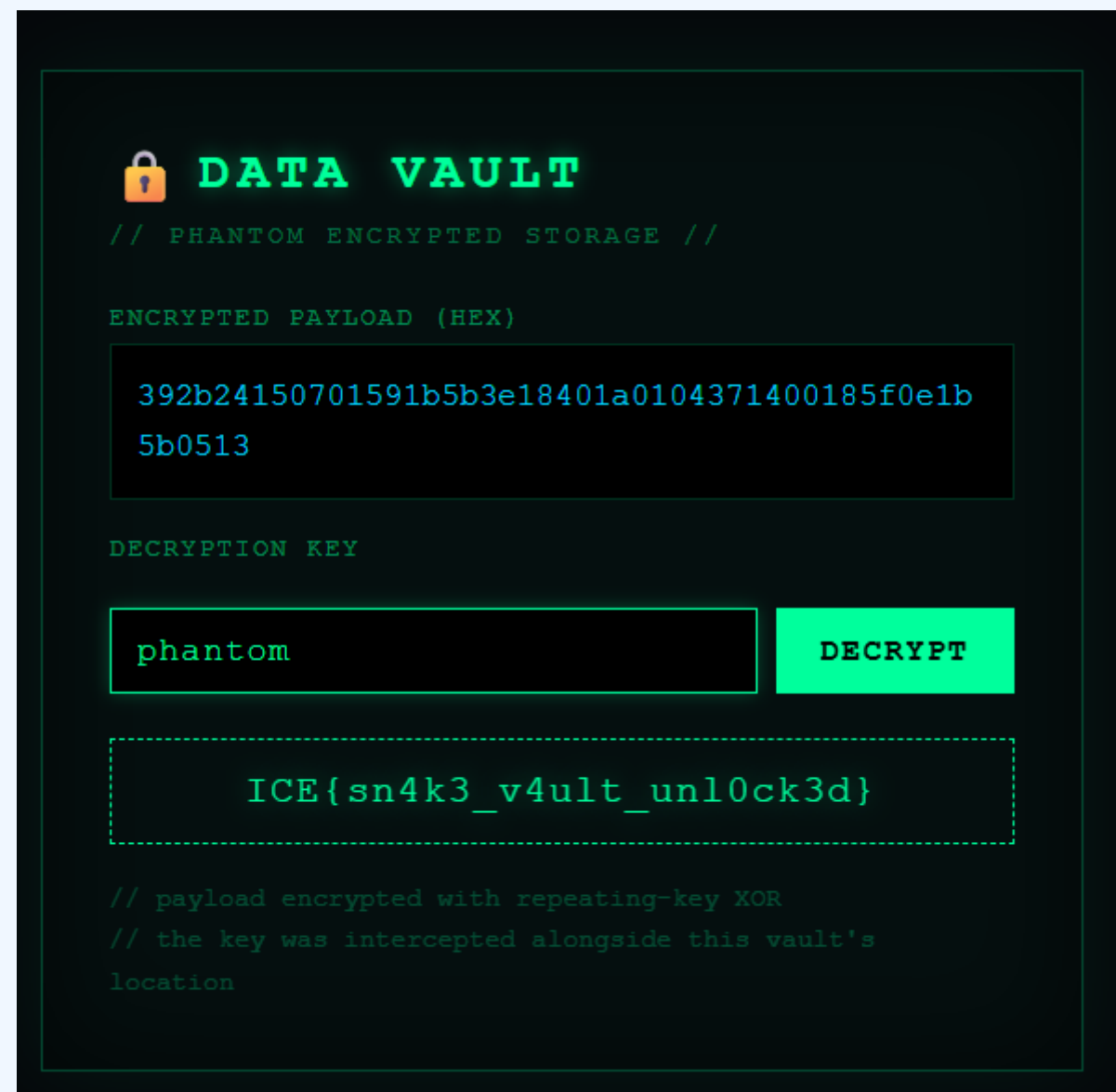
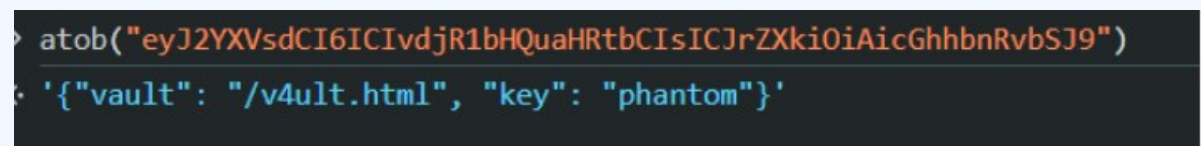
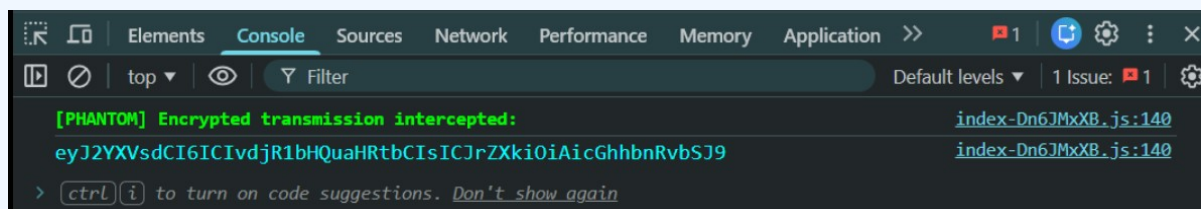
- 1 Відкрито онлайн-гру <https://snake.cybericebox.com/>.
- 2 Відкрито DevTools браузера (F12) → вкладка Console.
- 3 Досягнуто необхідного результату в грі; у консолі з'явився Base64-рядок.
- 4 Рядок декодовано командою: `atob("...")` у консолі.
- 5 Отримано прихований шлях /v4ult.html та ключ phantom.
- 6 Перейдено до /v4ult.html, введено ключ — отримано прапор.

ПРАПОР

ICE{sn4k3_v4ult_unl0ck3d}

Скріншоти





Завдання 5 — «Примарний сервер»

Категорія: **Network Forensics / Encoding** Складність: **Hard**

Інструменти: [Wireshark](#) · [Base64 Decoder](#) · [Hex Decoder](#) · [NATO Phonetic Alphabet](#)

Опис завдання

Багаторівневе кодування (Base64 → Hex → НАТО-алфавіт → акронім) є прикладом стеганографії на рівні протоколу. Зловмисники можуть використовувати подібні техніки для приховування C2-команд у, здавалося б, нешкідливому HTTP-трафіку.

Покрокове виконання

- 1 Відкрито файл s11Nat.pcapng у Wireshark, застосовано фільтр http.
- 2 У кількох POST-запитах знайдено Base64-кодовані фрагменти.
- 3 Кожен фрагмент декодовано з Base64, а результат — з Hex у ASCII.
- 4 Отримано слова фонетичного алфавіту НАТО: India, Charlie, Echo, Golf, Hotel, Oscar, Sierra, Tango.
- 5 Взято перші літери кожного слова: I, C, E, G, H, O, S, T.
- 6 Відновлено прапор з акроніму: ICE{GHOST}.

ПРАПОР

ICE {GHOST}

Скріншоти

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
Frame	100.0	571	100.0	375745	2,465	0	0	0	571
Ethernet	100.0	571	2.2	8438	55	0	0	0	571
Internet Protocol Version 6	0.4	2	0.0	80	0	0	0	0	2
Internet Control Message Protocol v6	0.4	2	0.0	16	0	2	16	0	2
Internet Protocol Version 4	97.2	555	3.0	11100	72	0	0	0	555
User Datagram Protocol	8.4	48	0.1	384	2	0	0	0	48
Domain Name System	8.4	48	1.0	3728	24	48	3728	24	48
Transmission Control Protocol	52.0	297	1.7	6228	40	236	5008	32	297
Transport Layer Security	7.5	43	13.5	50844	333	43	49453	324	44
Hypertext Transfer Protocol	3.2	18	1.5	5457	35	1	651	4	18
Line-based text data	1.4	8	74.5	279792	1,835	8	279792	1,835	8
HTML Form URL Encoded	1.6	9	0.0	103	0	9	103	0	9
Internet Control Message Protocol	36.8	210	3.6	13440	88	210	13440	88	210
Address Resolution Protocol	2.5	14	0.1	392	2	14	392	2	14

No.	Time	Source	Destination	Protocol	Length	Info
125	423.428622375	10.0.2.15	172.67.155.189	HTTP	217	POST / HTTP/1.1 (application/x-www-form-urlencoded)
140	463.377445777	10.0.2.15	128.59.105.24	HTTP	227	POST /~fdc/sample.html HTTP/1.1 (application/x-www-form-urlencoded)
167	558.986910713	10.0.2.15	128.59.105.24	HTTP	240	POST /~fdc/sample.html HTTP/1.1 (application/x-www-form-urlencoded)
196	646.103927199	10.0.2.15	128.59.105.24	HTTP	227	POST /~fdc/sample.html HTTP/1.1 (application/x-www-form-urlencoded)
223	683.650227920	10.0.2.15	128.59.105.24	HTTP	231	POST /~fdc/sample.html HTTP/1.1 (application/x-www-form-urlencoded)
252	941.645813305	10.0.2.15	128.59.105.24	HTTP	227	POST /~fdc/sample.html HTTP/1.1 (application/x-www-form-urlencoded)
279	993.298660046	10.0.2.15	128.59.105.24	HTTP	227	POST /~fdc/sample.html HTTP/1.1 (application/x-www-form-urlencoded)
387	1144.6938603	10.0.2.15	128.59.105.24	HTTP	227	POST /~fdc/sample.html HTTP/1.1 (application/x-www-form-urlencoded)
543	1214.9546394	10.0.2.15	128.59.105.24	HTTP	244	POST /~fdc/sample.html HTTP/1.1 (application/x-www-form-urlencoded)

```

> Ethernet II, Src: PCSSystemtec_53:0c:ba (08:00:27:53:0c:ba), Dst: 52:54:00:12:35:
> Internet Protocol Version 4, Src: 10.0.2.15, Dst: 128.59.105.24
> Transmission Control Protocol, Src Port: 57150, Dst Port: 80, Seq: 1, Ack: 1, Len
> Hypertext Transfer Protocol
  > POST /~fdc/sample.html HTTP/1.1\r\n
    Request Method: POST
    Request URI: /~fdc/sample.html
    Request Version: HTTP/1.1
    Host: www.columbia.edu\r\n
    User-Agent: curl/7.88.1\r\n
    Accept: */*\r\n
  > Content-Length: 8\r\n
    [Content length: 8]
    Content-Type: application/x-www-form-urlencoded\r\n
    \r\n
    [Response in frame: 154]
    [Full request URI: http://www.columbia.edu/~fdc/sample.html]
    File Data: 8 bytes
  > HTML Form URL Encoded: application/x-www-form-urlencoded
    > Form item: "SW5kaWE" = ""
      Key: SW5kaWE
      Value:

```

```

> Ethernet II, Src: PCSSystemtec_53:0c:ba (08:00:27:53:0c:ba), Dst: 52:54:00:12:35:
> Internet Protocol Version 4, Src: 10.0.2.15, Dst: 128.59.105.24
> Transmission Control Protocol, Src Port: 51814, Dst Port: 80, Seq: 1, Ack: 1, Len
> Hypertext Transfer Protocol
  > POST /~fdc/sample.html HTTP/1.1\r\n
    Request Method: POST
    Request URI: /~fdc/sample.html
    Request Version: HTTP/1.1
    Host: www.columbia.edu\r\n
    User-Agent: curl/7.88.1\r\n
    Accept: */*\r\n
  > Content-Length: 8\r\n
    [Content length: 8]
    Content-Type: application/x-www-form-urlencoded\r\n
    \r\n
    [Response in frame: 210]
    [Full request URI: http://www.columbia.edu/~fdc/sample.html]
    File Data: 8 bytes
  > HTML Form URL Encoded: application/x-www-form-urlencoded
    > Form item: "RWNobw" = "="
      Key: RWNobw
      Value: =

```

```

> Ethernet II, Src: PCSSystemtec_53:0c:ba (08:00:27:53:0c:ba), Dst: 52:54:00:12:35:
> Internet Protocol Version 4, Src: 10.0.2.15, Dst: 128.59.105.24
> Transmission Control Protocol, Src Port: 54894, Dst Port: 80, Seq: 1, Ack: 1, Len
> Hypertext Transfer Protocol
  > POST /~fdc/sample.html HTTP/1.1\r\n
    Request Method: POST
    Request URI: /~fdc/sample.html
    Request Version: HTTP/1.1
    Host: www.columbia.edu\r\n
    User-Agent: curl/7.88.1\r\n
    Accept: */*\r\n
  > Content-Length: 11\r\n
    [Content length: 11]
    Content-Type: application/x-www-form-urlencoded\r\n
    \r\n
    [Response in frame: 237]
    [Full request URI: http://www.columbia.edu/~fdc/sample.html]
    File Data: 11 bytes
  > HTML Form URL Encoded: application/x-www-form-urlencoded
    > Form item: "47 6f 6c 66" = ""
      Key: 47 6f 6c 66
      Value:

```

Request: Boolean

```

> Ethernet II, Src: PCSSystemtec_53:0c:ba (08:00:27:53:0c:ba), Dst: 52:54:00:12:35:
> Internet Protocol Version 4, Src: 10.0.2.15, Dst: 128.59.105.24
> Transmission Control Protocol, Src Port: 58688, Dst Port: 80, Seq: 1, Ack: 1, Len
> Hypertext Transfer Protocol
  > POST /~fdc/sample.html HTTP/1.1\r\n
    Request Method: POST
    Request URI: /~fdc/sample.html
    Request Version: HTTP/1.1
    Host: www.columbia.edu\r\n
    User-Agent: curl/7.88.1\r\n
    Accept: */*\r\n
  > Content-Length: 8\r\n
    [Content length: 8]
    Content-Type: application/x-www-form-urlencoded\r\n
    \r\n
    [Response in frame: 266]
    [Full request URI: http://www.columbia.edu/~fdc/sample.html]
    File Data: 8 bytes
  > HTML Form URL Encoded: application/x-www-form-urlencoded
    > Form item: "SG90ZWw" = ""
      Key: SG90ZWw
      Value:

```

```
>>> import base64
...
... fragments = [
...     ("140", "Base64", "SW5kaWE=", base64.b64decode("SW5kaWE=").decode()),
...     ("167", "Hex", "43 68 61 72 6c 69 65", bytes.fromhex("43 68 61 72 6c 69 65").decode()),
...     ("196", "Base64", "RWNoBw==", base64.b64decode("RWNoBw==").decode()),
...     ("223", "Hex", "47 6f 6c 66", bytes.fromhex("47 6f 6c 66").decode()),
...     ("252", "Base64", "SG90ZWw=", base64.b64decode("SG90ZWw=").decode()),
...     ("279", "Base64", "T3NjYXI=", base64.b64decode("T3NjYXI=").decode()),
...     ("387", "Base64", "U2llcnJh", base64.b64decode("U2llcnJh").decode()),
...     ("543", "Hex", "54 61 6e 67 6f", bytes.fromhex("54 61 6e 67 6f").decode()),
... ]
...
... letters = ""
...
... for packet, method, encoded, decoded in fragments:
...     letters += decoded[0]
...     print(f"Packet {packet}: {method:6} → {decoded} → {decoded[0]}")
...
... print()
... print("Reconstructed sequence:", letters)
... print("Flag: ICE{" + letters[3:] + "}")
...
Packet 140: Base64 → India → I
Packet 167: Hex → Charlie → C
Packet 196: Base64 → Ech → E
Packet 223: Hex → Golf → G
Packet 252: Base64 → Hotel → H
Packet 279: Base64 → Oscar → O
Packet 387: Base64 → Sierra → S
Packet 543: Hex → Tango → T

Reconstructed sequence: ICEGHOST
Flag: ICE{GHOST}
>>> █
```

Завдання 6 — «ICMP-відлуння»

Категорія: **Network Forensics / TTL Analysis** Складність: **Medium**

Інструменти: [Wireshark](#) · [ICMP Filter \(icmp.type == 0\)](#)

Опис завдання

Техніка TTL-аналізу дозволяє визначити кількість хопів до цільового хоста. Кожен маршрутизатор на шляху зменшує TTL на 1; якщо пакет досяг мети зі значенням 13, це і є відповіддю-прапором. Подібний підхід використовується в утиліті *tracert*.

Покрокове виконання

- 1 Відкрито файл icmp-ttl.pcap у Wireshark, застосовано фільтр icmp.
- 2 Виявлено серію ICMP Echo Request з поступово зростаючим TTL.
- 3 Пакети з малим TTL отримували відповідь «Time-to-live exceeded» від проміжних маршрутизаторів.
- 4 Застосовано фільтр icmp.type == 0 для пошуку першої успішної відповіді Echo Reply.
- 5 Через поле «Request frame» знайдено відповідний вихідний запит.
- 6 У IPv4-заголовку запиту значення TTL на момент досягнення цілі — 13.

ПРАПОР **ICSE{13}**

Скріншоти

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=768/3, ttl=1 (no response found!)
2	3.364382	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=1024/4, ttl=1 (no response found!)
3	6.368126	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=1280/5, ttl=1 (no response found!)
4	9.371764	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=1536/6, ttl=2 (no response found!)
5	12.383904	24.6.126.218	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
6	9.394103	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=1792/7, ttl=2 (no response found!)
7	9.407045	12.244.25.161	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
8	9.407187	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=2048/8, ttl=2 (no response found!)
9	9.433282	12.244.25.161	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
10	15.040775	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=2304/9, ttl=3 (no response found!)
11	15.005589	12.244.25.161	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
12	15.006158	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=2560/10, ttl=3 (no response found!)
13	15.076118	12.244.25.161	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
14	15.076262	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=2816/11, ttl=3 (no response found!)
15	15.098114	12.244.25.161	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
16	20.819234	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=3072/12, ttl=4 (no response found!)
17	20.832207	12.244.25.161	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
18	20.832483	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=3328/13, ttl=4 (no response found!)
19	20.853357	12.244.25.161	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
20	20.853611	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=3584/14, ttl=4 (no response found!)
21	20.869107	12.244.25.161	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
22	26.399007	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=3840/15, ttl=5 (no response found!)
23	26.419143	12.19287239	24.6.126.218	ICMP	106	Time-to-live exceeded (Time to live exceeded in transit)
24	26.419435	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=4096/16, ttl=5 (no response found!)
25	26.439858	12.122.2.250	24.6.126.218	ICMP	134	Time-to-live exceeded (Time to live exceeded in transit)
26	26.439996	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=4352/17, ttl=5 (no response found!)
27	26.461652	12.122.2.250	24.6.126.218	ICMP	134	Time-to-live exceeded (Time to live exceeded in transit)
28	27.472452	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=4608/18, ttl=6 (no response found!)
29	27.497676	12.123.28.129	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
30	27.498027	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=4864/19, ttl=6 (no response found!)
31	27.528548	12.123.28.129	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
32	27.528748	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=5120/20, ttl=6 (no response found!)
33	27.560144	12.123.28.129	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
34	28.622120	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=5376/21, ttl=7 (no response found!)
35	28.653264	129.250.9.109	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
36	28.653466	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=5632/22, ttl=7 (no response found!)
37	28.673235	129.250.9.109	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
38	28.673425	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=5888/23, ttl=7 (no response found!)
39	28.708714	129.250.9.109	24.6.126.218	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
40	34.285156	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=6144/24, ttl=8 (no response found!)
41	34.387675	129.250.2.112	24.6.126.218	ICMP	182	Time-to-live exceeded (Time to live exceeded in transit)
42	34.387968	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=6400/25, ttl=8 (no response found!)
43	34.392477	129.250.2.112	24.6.126.218	ICMP	182	Time-to-live exceeded (Time to live exceeded in transit)
44	34.392820	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=6656/26, ttl=8 (no response found!)
45	34.385728	129.250.2.112	24.6.126.218	ICMP	182	Time-to-live exceeded (Time to live exceeded in transit)
46	35.759869	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=6912/27, ttl=9 (no response found!)
47	35.822520	129.250.4.197	24.6.126.218	ICMP	182	Time-to-live exceeded (Time to live exceeded in transit)
48	35.822658	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=7168/28, ttl=9 (no response found!)
49	35.810630	129.250.4.197	24.6.126.218	ICMP	182	Time-to-live exceeded (Time to live exceeded in transit)
50	35.876783	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=7424/29, ttl=9 (no response found!)
51	35.926870	129.250.4.197	24.6.126.218	ICMP	182	Time-to-live exceeded (Time to live exceeded in transit)
52	35.950503	24.6.126.218	198.173.244.32	ICMP	106	Echo (ping) request id=0x0200, seq=7680/30, ttl=10 (no response found!)

No.	Time	Source	Destination	Protocol	Length	Info
72	43.934598	198.173.244.32	24.6.126.218	ICMP	106	Echo (ping) reply id=0x0200, seq=9984/39, ttl=51 (request in 74)
74	49.424540	198.173.244.32	24.6.126.218	ICMP	106	Echo (ping) reply id=0x0200, seq=10240/40, ttl=51 (request in 73)
76	49.503822	198.173.244.32	24.6.126.218	ICMP	106	Echo (ping) reply id=0x0200, seq=10496/41, ttl=51 (request in 75)

```

> Frame 72: Packet, 106 bytes on wire (848 bits), 106 bytes captured (848 bits)
> Ethernet II, Src: Cadant_22:89:c2 (00:01:5c:22:89:c2), Dst: AmbitMicrosy_aa:af:80
> Internet Protocol Version 4, Src: 198.173.244.32, Dst: 24.6.126.218
> Internet Control Message Protocol
  Type: Echo (ping) reply (0)
  Code: 0
  Checksum: 0xd6ff [correct]
  [Checksum Status: Good]
  Identifier (BE): 512 (0x0200)
  Identifier (LE): 2 (0x0002)
  Sequence Number (BE): 9984 (0x2700)
  Sequence Number (LE): 39 (0x0027)
  [Request frame: 71]
  [Response time: 93.110 ms]
  Data (64 bytes)

```

Завдання 7 — «Загадковий сайт»

Категорія: Web / API Security

Складність: Medium

Інструменти: Web Browser · View Page Source · Browser Address Bar · Base64 (atob())

Опис завдання

Задача ілюструє вразливість типу API Information Disclosure (OWASP API3). Застарілі endpoint'и, залишені у продакшн-середовищі, можуть розкривати внутрішню документацію та конфігурацію. Відкритий `/api/v2/config` без автентифікації є критичною вразливістю у реальному середовищі.

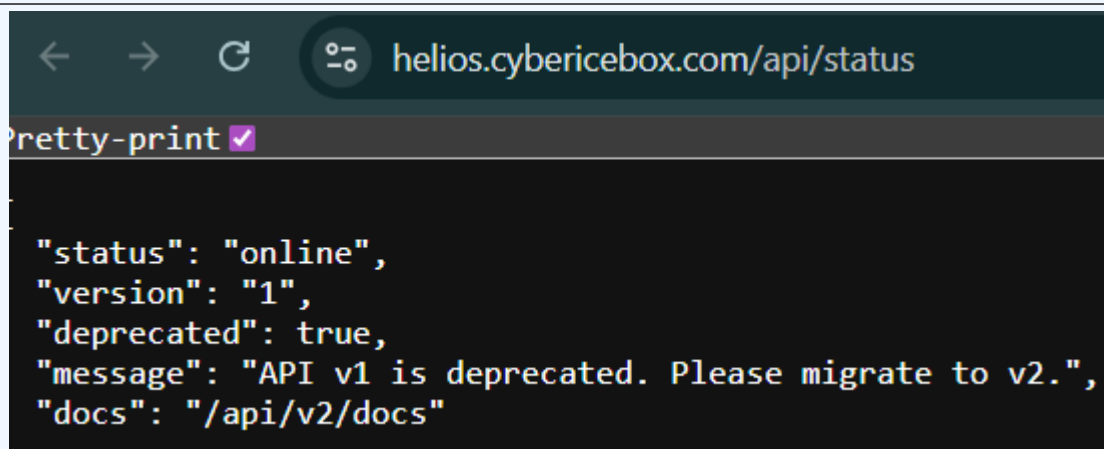
Покрокове виконання

- 1 Відкрито сайт `https://helios.cybericebox.com`, переглянуто вихідний HTML.
- 2 У коментарях та JS знайдено застарілий маршрут `/api/status`.
- 3 Перехід до `/api/status` повернув посилання на `/api/v2/docs` (документація API v2).
- 4 У документації виявлено відкритий endpoint `/api/v2/config` без авторизації.
- 5 JSON-відповідь містила системний токен у форматі Base64.
- 6 Декодування `atob()` повернуло прапор.

ПРАПОР

ICE{4p1_134k}

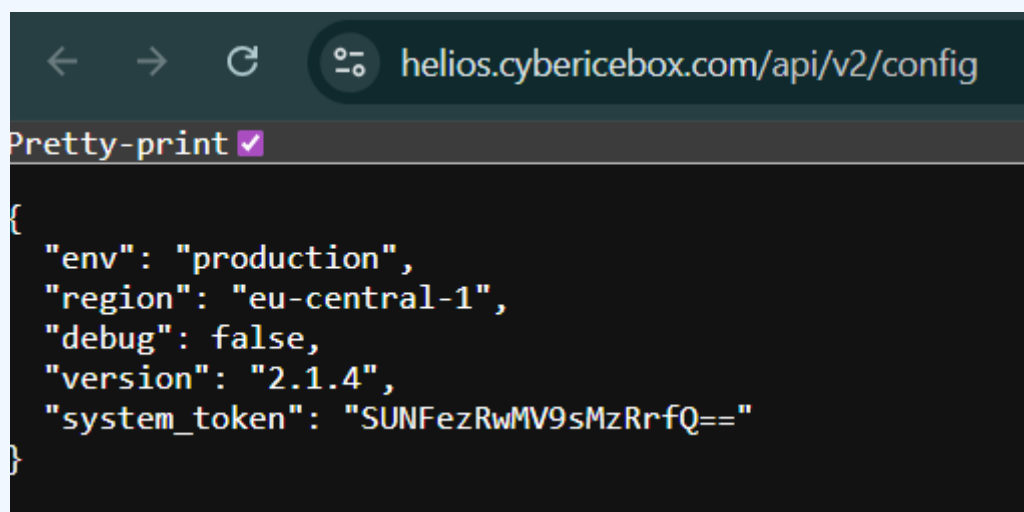
Скріншоти



```
helios.cybericebox.com/api/status

pretty-print ✓

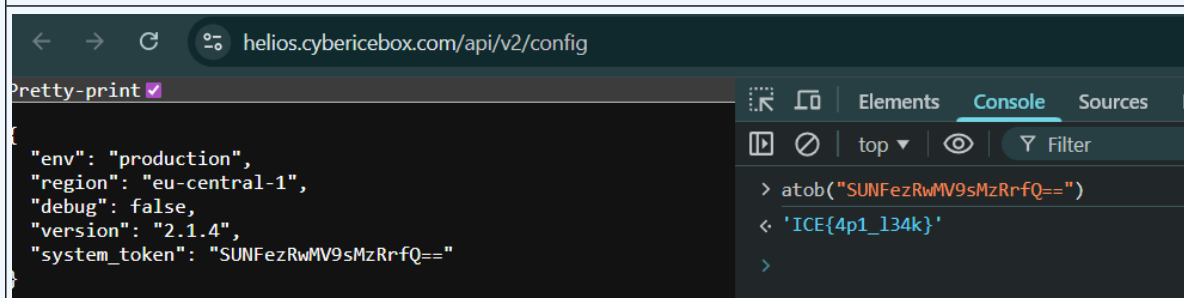
{
  "status": "online",
  "version": "1",
  "deprecated": true,
  "message": "API v1 is deprecated. Please migrate to v2.",
  "docs": "/api/v2/docs"
}
```



```
helios.cybericebox.com/api/v2/config

Pretty-print ✓

{
  "env": "production",
  "region": "eu-central-1",
  "debug": false,
  "version": "2.1.4",
  "system_token": "SUNFezRwMV9sMzRrfQ=="
}
```



```
helios.cybericebox.com/api/v2/config

Pretty-print ✓

{
  "env": "production",
  "region": "eu-central-1",
  "debug": false,
  "version": "2.1.4",
  "system_token": "SUNFezRwMV9sMzRrfQ=="
}

> atob("SUNFezRwMV9sMzRrfQ==")
< 'ICE{4p1_l34k}'
>
```


Завдання 8 — «Зашифрований кадр»

Категорія: **Forensics / Binary Analysis** Складність: **Hard**

Інструменти: [Kali Linux Terminal](#) · [xxd](#) · [strings](#) · [Base64](#) · [Hex Decoder](#)

Опис завдання

Завдання відтворює реальний сценарій forensic-аналізу: відновлення мережевого кадру з текстового Нех-представлення та видобування секретних даних з HTTP-заголовків. Двошарове кодування (Base64 → Нех) ускладнює виявлення токена при поверхневому аналізі.

Покрокове виконання

- 1 Завантажено файл frame.txt, виявлено текстовий Нех-дамп байтів.
- 2 Команда `xxd -r -p frame.txt > frame.bin` конвертувала Нех у бінарний файл.
- 3 Команда `strings frame.bin` виявила HTTP POST-запит із заголовком X-Auth-Token.
- 4 Значення токена декодовано з Base64.
- 5 Отриманий бінарний результат додатково конвертовано з Нех у ASCII.
- 6 Отримано прапор `ICE{fr4me_d3c0d3d}`.

ПРАПОР `ICE{fr4me_d3c0d3d}`

Скріншоти

```
(kali@kali)-[~/Desktop]
$ file frame.txt
frame.txt: ASCII text

(kali@kali)-[~/Desktop]
$ wc -c frame.txt
786 frame.txt

(kali@kali)-[~/Desktop]
$ wc -l frame.txt
17 frame.txt

(kali@kali)-[~/Desktop]
$

(kali@kali)-[~/Desktop]
$ sed -n '1,20p' frame.txt
00 1B 44 11 3A B7 00 05 73 A0 00 00 08 00 45 00
00 F8 43 21 40 00 40 06 2A FE C0 A8 01 32 0A 00
00 07 C0 A9 1F 90 1A 2B 3C 4D 5E 6F 7A 8B 50 18
72 10 E9 49 00 00 50 4F 53 54 20 2F 61 70 69 2F
75 70 6C 6F 61 64 20 48 54 54 50 2F 31 2E 31 0D
0A 48 6F 73 74 3A 20 70 68 61 6E 74 6F 6D 2D 6E
6F 64 65 2E 6C 6F 63 61 6C 0D 0A 55 73 65 72 2D
41 67 65 6E 74 3A 20 50 68 61 6E 74 6F 6D 41 67
65 6E 74 2F 31 2E 30 0D 0A 58 2D 41 75 74 68 2D
54 6F 68 65 6E 3A 20 4E 44 68 30 4D 7A 51 31 4E
32 49 32 4E 6A 63 79 4D 7A 51 32 5A 44 59 31 4E
57 59 32 4E 44 4D 7A 4E 6A 4D 7A 4D 44 59 30 4D
7A 4D 32 4E 44 64 68 0D 0A 43 6F 6E 74 65 6E 74
2D 54 79 70 65 3A 20 61 70 70 6C 69 63 61 74 69
6F 6E 2F 6F 63 74 65 74 2D 73 74 72 65 61 6D 0D
0A 43 6F 6E 74 65 6E 74 2D 4C 65 6E 67 74 68 3A
20 30 0D 0A 0D 0A
```

```
(kali@kali)-[~/Desktop]
$ head -c 500 frame.txt
00 1B 44 11 3A B7 00 05 73 A0 00 00 08 00 45 00
00 F8 43 21 40 00 40 06 2A FE C0 A8 01 32 0A 00
00 07 C0 A9 1F 90 1A 2B 3C 4D 5E 6F 7A 8B 50 18
72 10 E9 49 00 00 50 4F 53 54 20 2F 61 70 69 2F
75 70 6C 6F 61 64 20 48 54 54 50 2F 31 2E 31 0D
0A 48 6F 73 74 3A 20 70 68 61 6E 74 6F 6D 2D 6E
6F 64 65 2E 6C 6F 63 61 6C 0D 0A 55 73 65 72 2D
41 67 65 6E 74 3A 20 50 68 61 6E 74 6F 6D 41 67
65 6E 74 2F 31 2E 30 0D 0A 58 2D 41 75 74 68 2D
54 6F 6B 65 6E 3A 20 4E 44 6B 30 4D 7A 51 31 4E
32 49 32 4E 6A 63 79
```

```
(kali@kali)-[~/Desktop]
$ cat -A frame.txt
00 1B 44 11 3A B7 00 05 73 A0 00 00 08 00 45 00$
00 F8 43 21 40 00 40 06 2A FE C0 A8 01 32 0A 00$
00 07 C0 A9 1F 90 1A 2B 3C 4D 5E 6F 7A 8B 50 18$
72 10 E9 49 00 00 50 4F 53 54 20 2F 61 70 69 2F$
75 70 6C 6F 61 64 20 48 54 54 50 2F 31 2E 31 0D$
0A 48 6F 73 74 3A 20 70 68 61 6E 74 6F 6D 2D 6E$
6F 64 65 2E 6C 6F 63 61 6C 0D 0A 55 73 65 72 2D$
41 67 65 6E 74 3A 20 50 68 61 6E 74 6F 6D 41 67$
65 6E 74 2F 31 2E 30 0D 0A 58 2D 41 75 74 68 2D$
54 6F 6B 65 6E 3A 20 4E 44 6B 30 4D 7A 51 31 4E$
32 49 32 4E 6A 63 79 4D 7A 51 32 5A 44 59 31 4E$
57 59 32 4E 44 4D 7A 4E 6A 4D 7A 4D 44 59 30 4D$
7A 4D 32 4E 44 64 6B 0D 0A 43 6F 6E 74 65 6E 74$
2D 54 79 70 65 3A 20 61 70 70 6C 69 63 61 74 69$
6F 6E 2F 6F 63 74 65 74 2D 73 74 72 65 61 6D 0D$
0A 43 6F 6E 74 65 6E 74 2D 4C 65 6E 67 74 68 3A$
20 30 0D 0A 0D 0A$
```

```
(kali@kali)-[~/Desktop]
$ strings -a frame.txt
00 1B 44 11 3A B7 00 05 73 A0 00 00 08 00 45 00
00 F8 43 21 40 00 40 06 2A FE C0 A8 01 32 0A 00
00 07 C0 A9 1F 90 1A 2B 3C 4D 5E 6F 7A 8B 50 18
72 10 E9 49 00 00 50 4F 53 54 20 2F 61 70 69 2F
75 70 6C 6F 61 64 20 48 54 54 50 2F 31 2E 31 0D
0A 48 6F 73 74 3A 20 70 68 61 6E 74 6F 6D 2D 6E
6F 64 65 2E 6C 6F 63 61 6C 0D 0A 55 73 65 72 2D
41 67 65 6E 74 3A 20 50 68 61 6E 74 6F 6D 41 67
65 6E 74 2F 31 2E 30 0D 0A 58 2D 41 75 74 68 2D
54 6F 6B 65 6E 3A 20 4E 44 6B 30 4D 7A 51 31 4E
32 49 32 4E 6A 63 79 4D 7A 51 32 5A 44 59 31 4E
57 59 32 4E 44 4D 7A 4E 6A 4D 7A 4D 44 59 30 4D
7A 4D 32 4E 44 64 6B 0D 0A 43 6F 6E 74 65 6E 74
2D 54 79 70 65 3A 20 61 70 70 6C 69 63 61 74 69
6F 6E 2F 6F 63 74 65 74 2D 73 74 72 65 61 6D 0D
0A 43 6F 6E 74 65 6E 74 2D 4C 65 6E 67 74 68 3A
20 30 0D 0A 0D 0A
```

```
(kali@kali)-[~/Desktop]
$ strings -a frame.txt | grep -Ein 'ICE\\[!flag|key|secret|token|phantom'
```

```
(kali@kali)-[~/Desktop]
$
```

```
(kali@kali)-[~/Desktop]
$ wc -c frame.bin
262 frame.bin

(kali@kali)-[~/Desktop]
$ strings -a frame.bin
+<M"oz
POST /api/upload HTTP/1.1
Host: phantom-node.local
User-Agent: PhantomAgent/1.0
X-Auth-Token: NDk0MzQ1N2I2NjcyMzQ2ZDY1NWY2NDMzMjMzMDY0MzMDNDdk
Content-Type: application/octet-stream
Content-Length: 0
```

```
(kali@kali)-[~/Desktop]
$
```

Завдання 9 — «Перший слід»

Категорія: Network Forensics / URL Encoding Складність: Easy

Інструменти: Wireshark · HTTP Filter · URL Decoder

Опис завдання

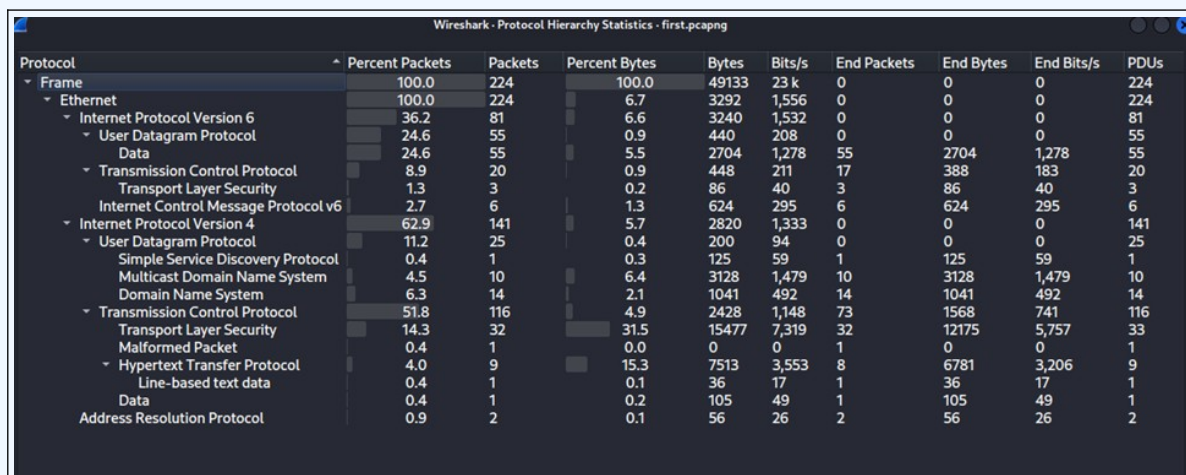
POST-запити з формою входу передавали облікові дані у відкритому HTTP-трафіку. Прапор містився в параметрі email у форматі URL Encoding. Це демонструє ризик використання HTTP (замість HTTPS): будь-який перехоплювач у мережі може прочитати паролі та інші чутливі дані.

Покрокове виконання

- 1 Відкрито файл first.pcapng у Wireshark.
- 2 Проаналізовано структуру трафіку через Statistics → Protocol Hierarchy.
- 3 Застосовано фільтр http та переглянуто POST-запити.
- 4 У пакеті №96 знайдено POST до /ajax/login/login.ashx.
- 5 Параметр email містив рядок ICE%7BaKuL1edo3%7D.
- 6 URL-декодування (%7B → {, %7D → }) дало прапор ICE{aKuL1edo3}.

ПРАПОР ICE{aKuL1edo3}

Скріншоти



Wireshark - Protocol Hierarchy Statistics - first.pcapng

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	224	100.0	49133	23 k	0	0	0	224
Ethernet	100.0	224	6.7	3292	1,556	0	0	0	224
Internet Protocol Version 6	36.2	81	6.6	3240	1,532	0	0	0	81
User Datagram Protocol	24.6	55	0.9	440	208	0	0	0	55
Data	24.6	55	5.5	2704	1,278	55	2704	1,278	55
Transmission Control Protocol	8.9	20	0.9	448	211	17	388	183	20
Transport Layer Security	1.3	3	0.2	86	40	3	86	40	3
Internet Control Message Protocol v6	2.7	6	1.3	624	295	6	624	295	6
Internet Protocol Version 4	62.9	141	5.7	2820	1,333	0	0	0	141
User Datagram Protocol	11.2	25	0.4	200	94	0	0	0	25
Simple Service Discovery Protocol	0.4	1	0.3	125	59	1	125	59	1
Multicast Domain Name System	4.5	10	6.4	3128	1,479	10	3128	1,479	10
Domain Name System	6.3	14	2.1	1041	492	14	1041	492	14
Transmission Control Protocol	51.8	116	4.9	2428	1,148	73	1568	741	116
Transport Layer Security	14.3	32	31.5	15477	7,319	32	12175	5,757	33
Malformed Packet	0.4	1	0.0	0	0	1	0	0	1
Hypertext Transfer Protocol	4.0	9	15.3	7513	3,553	8	6781	3,206	9
Line-based text data	0.4	1	0.1	36	17	1	36	17	1
Data	0.4	1	0.2	105	49	1	105	49	1
Address Resolution Protocol	0.9	2	0.1	56	26	2	56	26	2

No.	Time	Source	Destination	Protocol	Length	Info
96	7.654887	192.168.0.252	163.181.92.232	HTTP	979	POST /ajax/login/login.ashx?email=ICE%7BaKuL1edo3%7D&pws=cf39743937cae11ab4e5eae3e11158...
97	7.655040	192.168.0.252	163.181.131.235	HTTP	671	GET /pageclick.jpg?code=64-4-1&islogin=0&uid=1730996229966-7 HTTP/1.1
107	8.087894	163.181.131.235	192.168.0.252	HTTP	573	HTTP/1.1 200 OK
114	8.341247	163.181.92.232	192.168.0.252	HTTP	822	HTTP/1.1 200 OK (text/plain)
178	11.986696	192.168.0.252	163.181.92.232	HTTP	995	GET /ajax/index7/getZZCData.ashx?artNum=20&nPage=1&nId=1078282137&isNewArt=true&sign=7...
187	12.752036	163.181.92.232	192.168.0.252	HTTP	1309	Continuation
188	12.752036	163.181.92.232	192.168.0.252	HTTP	189	[TCP Previous segment not captured] Continuation
190	12.752224	163.181.92.232	192.168.0.252	HTTP	1466	[TCP Out-Of-Order] Continuation
192	12.753473	163.181.92.232	192.168.0.252	HTTP	1634	Continuation

```

Frame 96: Packet, 979 bytes on wire (7832 bits), 979 bytes captured (7832 bits) on
Ethernet II, Src: Intel_22:9f:20 (f8:e4:e3:22:9f:20), Dst: MS-NLB-PhysServer-16_1f
Internet Protocol Version 4, Src: 192.168.0.252, Dst: 163.181.92.232
Transmission Control Protocol, Src Port: 64099, Dst Port: 80, Seq: 1, Ack: 1, Len:
Hypertext Transfer Protocol
  POST /ajax/login/login.ashx?email=ICE%7BaKuL1edo3%7D&pws=cf39743937cae11ab4e5eae3e11158e5
    Request Method: POST
    Request URI: /ajax/login/login.ashx?email=ICE%7BaKuL1edo3%7D&pws=cf39743937cae11ab4e5eae3e11158e5
    Request URI Path: /ajax/login/login.ashx
    Request URI Query: email=ICE%7BaKuL1edo3%7D&pws=cf39743937cae11ab4e5eae3e11158e5
      Request URI Query Parameter: email=ICE%7BaKuL1edo3%7D
      Request URI Query Parameter: pws=cf39743937cae11ab4e5eae3e11158e5
      Request URI Query Parameter: isr=0
      Request URI Query Parameter: login=1
    Request Version: HTTP/1.1
    Host: www.360doc.com\r\n
    Connection: keep-alive\r\n
    Content-Length: 0\r\n

```

Завдання 10 — «Зсув у часі»

Категорія: Cryptography / Binary Analysis Складність: Hard

Інструменти: Kali Linux Terminal · xxd · Python 3 · file

Опис завдання

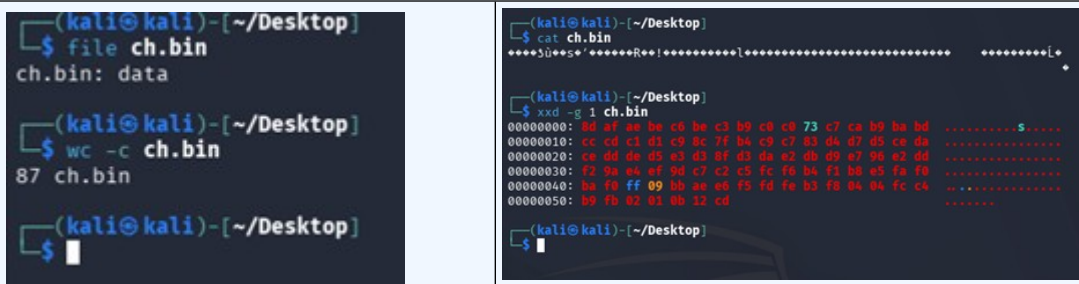
Byte-shift cipher зі змінним кроком є нестандартним шифром потоку. На відміну від простого ROT13/Caesar, кожен наступний байт зсувається на +1 більше від попереднього. Brute-force перебір початкового значення з перевіркою читабельності виводу — стандартний підхід для злому таких шифрів.

Покрокове виконання

- 1 Завантажено файл ch.bin; команда file ідентифікувала його як бінарні дані.
- 2 Перегляд через xxd показав послідовність байтів, що не відповідають ASCII.
- 3 Підказка про «зсув» вказала на byte-shift cipher зі змінним кроком.
- 4 Написано Python-скрипт для перебору початкових значень зсуву (0–255).
- 5 Враховано збільшення зсуву на кожній наступній позиції (+1 на кожен байт).
- 6 При правильному початковому зсуві скрипт відновив читабельне повідомлення з прапором.

ПРАПОР ICE{t1m3_shift}

Скріншоти



```
(kali@kali)-[~/Desktop]
$ file ch.bin
ch.bin: data

(kali@kali)-[~/Desktop]
$ wc -c ch.bin
87 ch.bin

(kali@kali)-[~/Desktop]
$ xxd -g 1 ch.bin
00000000: 8d af ae be c0 be c3 b9 c8 c0 73 c7 ca b9 ba bd .....$.....
00000010: cc c9 d1 d1 c9 bc 7f b4 c9 c7 a3 d4 d7 d5 ce da .....
00000020: ce d3 de d5 d3 d3 d3 d3 d3 d3 d3 d3 d3 d3 d3 d3 .....
00000030: f2 ba e4 ef 9d c7 c2 c5 fc f6 b4 f3 b8 e5 f6 f8 .....
00000040: ba f9 ff 09 bb ae e6 f5 fd fe b3 f8 04 04 fc c4 .....
00000050: b9 fb 02 01 0b 12 cd .....
```

```
Session Actions Edit View Help
GNU nano 8.7.1 solve_shift.py
import pathlib
data = Path("ch.bin").read_bytes()

modes = [
    "constant_shift": lambda byte, index, start: (byte - start) % 256,
    "progressive_shift": lambda byte, index, start: (byte - start - index) % 256,
    "reverse_progressive_shift": lambda byte, index, start: (byte - start + index) % 256,
]

found = False

for mode_name, decode_byte in modes.items():
    for initial_shift in range(255):
        decoded = bytes(
            decode_byte(byte, index, initial_shift)
            for index, byte in enumerate(data)
        )
        if b"ICE{" in decoded:
            found = True
            print(f"[+] Mode: {mode_name}")
            print(f"[+] Initial shift: {initial_shift}")
            print(decoded.decode("ascii", errors="replace"))
            print()

if not found:
    print("No candidate containing ICE{ was found.")
```

```
Session Actions Edit View Help
kali@kali: ~/Desktop
$ nano solve_shift.py
kali@kali: ~/Desktop
$ python3 solve_shift.py
[+] Mode: progressive_shift
[+] Initial shift: 73
Decryption successful. The progressive cipher key is ICE{tim3_shift}. Well done, agent.
kali@kali: ~/Desktop
$
```


Завдання 11 — «Фінальна головоломка»

Категорія: **Cryptography / Reverse Engineering** Складність: **Hard**

Інструменти: [Python 3](#) · [base64 module](#) · [XOR operations](#)

Опис завдання

Фінальне завдання вимагало реверс-інжинірингу Python-коду та розуміння комбінованих криптографічних операцій. XOR-шифрування з відомим або відновлюваним ключем є оборотною операцією: повторне XOR з тим самим ключем дає оригінальні дані. Підказка PHANTOM була вбудована в умові задачі.

Покрокове виконання

- 1 Завантажено файли main.py та output.txt, проаналізовано логіку програми.
- 2 Встановлено ланцюжок кодування: XOR(key) → reverse bytes → Base64.
- 3 Ключем до відновлення стало кодове ім'я угруповання PHANTOM (з умови задачі).
- 4 Написано Python-скрипт для зворотнього розшифрування.
- 5 Крок 1: Base64-декодування output.txt.
- 6 Крок 2: Реверс отриманих байтів.
- 7 Крок 3: XOR із ключем PHANTOM — отримано фінальний прапор.

ПРАПОР ICE{Ph4nt0m_unm4sk3d_2026}

Скріншоти

```
(kali@kali)-[~/Desktop]
$ cat main.py
import base64

def xor(data: bytes, key: bytes) -> bytes:
    return bytes(data[i] ^ key[i % len(key)] for i in range(len(data)))

def main():
    flag = input("Enter the flag: ").encode()
    key = input("Enter the secret key: ").encode()

    data = xor(flag, key)
    data = data[::-1]
    data = base64.b64encode(data)

    print(data.decode())

main()

(kali@kali)-[~/Desktop]
$
```

```
(kali@kali)-[~/Desktop]
$ cat output.txt
KXhzeGISK2cIMnw9IzoLI3E8PnknBDUECk=

(kali@kali)-[~/Desktop]
$
```



```
(kali@kali)-[~/Desktop]
$ cat solve.py
import base64
from pathlib import Path

def xor(data: bytes, key: bytes) -> bytes:
    return bytes(
        data[index] ^ key[index % len(key)]
        for index in range(len(data))
    )

encoded = Path("output.txt").read_text().strip()
key = b"PHANTOM"

data = base64.b64decode(encoded)
data = data[::-1]
flag = xor(data, key)

print(flag.decode())

(kali@kali)-[~/Desktop]
$
```

```
(kali@kali)-[~/Desktop]
$ python3 solve.py
ICE{Ph4nt0m_unm4sk3d_2026}

(kali@kali)-[~/Desktop]
$
```